- ***2*** There are two sets of action-spaces we can import: one with 5 actions (simple) and one with 12 (complex).
- ***3*** Wraps the environment's action space to be 12 discrete actions
- ***4*** Tests the environment by taking random actions

If everything went well, a little window should pop up displaying Super Mario Bros., but it will be taking random actions and not making any forward progress through the level. By the end of this chapter you will have trained an agent that makes consistent forward progress and has learned to avoid or jump on enemies and to jump over obstacles. This, only using the intrinsic curiosity-based reward.

In the OpenAI Gym interface, the environment is instantiated as a class object called `env` and the main method you need to use is its `step(...)` method. The `step` method takes an integer representing the action to be taken. As with all OpenAI Gym environments, this one returns `state`, `reward`, `done`, and `info` data after each action is taken. The `state` is a numpy array with dimensions (240, 256, 3) representing an RGB video frame. The `reward` is bounded between –15 and 15 and is based on the amount of forward progress. The `done` variable is a Boolean that indicates whether or not the game is over (e.g., whether Mario dies). The `info` variable is a Python dictionary with the metadata listed in table 8.2.

Table 8.2. The metadata returned after each action in the `info` variable (source: https://github.com/Kautenja/gym-super-mario-bros)

Key	Type	Description
coins	int	The number of collected coins
flag_get	bool	True if Mario reached a flag or ax
life	int	The number of lives left, i.e., {3, 2, 1}
score	int	The cumulative in-game score
stage	int	The current stage, i.e., {1, ..., 4}
status	str	Mario's status, i.e., {'small', 'tall', 'fireball'}
time	int	The time left on the clock
world	int	The current world, i.e., {1, ..., 8}
x_pos	int	Mario's *x* position in the stage